

企业资源规划系统（ERP）设计文档

版本：v2.1.0
发布日期：2024年12月

目 录

第1章 引言	3
1.1 项目背景	3
1.2 项目目标	4
1.3 文档范围	5
第2章 总体架构设计	6
2.1 系统架构概述	6
2.1.1 分层架构设计	6
2.1.2 模块划分	7
2.2 技术选型	8
2.3 部署架构	9
第3章 功能模块设计	11
3.1 财务管理模块	11
3.2 人力资源模块	13
3.3 供应链管理模块	15
3.4 客户关系管理模块	17
第4章 数据库设计	19
4.1 数据库架构	19
4.2 核心数据表设计	20
第5章 接口设计	22
第6章 安全性设计	24
第7章 性能优化	26

第1章 引言

1.1 项目背景

随着企业规模的不断扩大和业务的日益复杂，传统的分散式管理系统已经无法满足企业发展的需求。各部门之间信息孤岛严重，数据共享困难，决策支持能力不足。为了提高企业的运营效率，降低管理成本，增强企业的核心竞争力，公司决定开发一套全新的企业资源规划（ERP）系统。

本ERP系统将整合企业的财务管理、人力资源管理、供应链管理、客户关系管理等核心业务模块，实现业务流程的标准化和信息资源的共享化，为企业管理层提供及时、准确、全面的决策支持。

1.2 项目目标

本项目的主要目标包括：建立统一的企业信息平台，消除信息孤岛；实现业务流程的自动化和标准化，提高运营效率30%以上；提供实时的数据分析和报表功能，支持科学决策；构建可扩展的系统架构，满足未来5年的业务发展需求；确保系统的安全性、稳定性和可靠性，达到99.9%的系统可用性。

1.3 文档范围

本文档详细描述了ERP系统的设计方案，包括总体架构设计、功能模块设计、数据库设计、接口设计、安全性设计和性能优化策略等内容。本文档的读者包括项目管理人员、系统架构师、开发人员、测试人员以及相关业务人员。

第2章 总体架构设计

2.1 系统架构概述

2.1.1 分层架构设计

本系统采用经典的分层架构模式，从上到下依次为：表现层、应用层、业务逻辑层、数据访问层和数据层。各层之间通过明确的接口进行通信，实现了关注点分离，提高了系统的可维护性和可扩展性。

表现层负责用户界面的展示和用户交互的处理；应用层负责协调业务逻辑的执行，实现业务流程的编排；业务逻辑层包含核心的业务规则和领域逻辑；数据访问层负责与数据库的交互，提供数据持久化服务；数据层则是数据的物理存储。

2.1.2 模块划分

系统划分为四大核心业务模块：财务管理模块、人力资源模块、供应链管理模块和客户关系管理模块。每个模块又进一步细分为多个子模块，各模块之间通过标准接口进行通信，实现了高内聚低耦合的设计原则。

2.2 技术选型

后端采用Java Spring Boot作为核心开发框架，使用MyBatis Plus作为ORM框架，数据库采用MySQL 8.0，缓存使用Redis，消息队列使用RabbitMQ。前端采用Vue 3 + TypeScript + Element Plus技术栈。微服务架构使用Spring Cloud Alibaba，服务注册与发现使用Nacos，配置中心使用Apollo。

2.3 部署架构

系统采用容器化部署方案，使用Docker进行应用打包，Kubernetes进行容器编排。部署环境分为开发环境、测试环境、预发布环境和生产环境。生产环境采用多可用区部署，确保系统的高可用性。使用Nginx作为反向代理和负载均衡器，ELK Stack进行日志收集和分析，Prometheus + Grafana进行系统监控。

第3章 功能模块设计

3.1 财务管理模块

财务管理模块是ERP系统的核心模块之一，包括总账管理、应收应付管理、固定资产管理、成本核算、预算管理和报表分析等子模块。该模块实现了财务数据的自动化处理，减少了人工操作，提高了财务数据的准确性和及时性。

总账管理模块支持多账簿、多币种核算，提供灵活的凭证管理和期末处理功能。应收应付管理模块实现了对客户和供应商往来账款的全面管理，支持账龄分析、坏账计提等功能。固定资产管理模块支持资产的全生命周期管理，包括资产购置、折旧、调拨、处置等。

3.2 人力资源模块

人力资源模块涵盖了企业人力资源管理的各个方面，包括组织管理、人事管理、考勤管理、薪酬管理、绩效管理、招聘管理和培训管理等子模块。该模块帮助企业实现人力资源管理的信息化、规范化和流程化。

3.3 供应链管理模块

供应链管理模块包括采购管理、库存管理、销售管理和物流管理等子模块。该模块实现了从采购到销售的完整供应链流程管理，帮助企业优化库存水平，提高供应链响应速度，降低运营成本。

3.4 客户关系管理模块

客户关系管理模块包括客户信息管理、销售机会管理、客户服务管理和数据分析等子模块。该模块帮助企业建立以客户为中心的经营理念，提高客户满意度和忠诚度，促进销售业绩的增长。

第4章 数据库设计

4.1 数据库架构

系统采用主从复制和读写分离的数据库架构。主库负责数据写入操作，多个从库负责数据读取操作，通过中间件实现读写路由。对于超大规模的数据表，采用分库分表方案，使用ShardingSphere进行数据分片。冷热数据分离存储，历史数据归档到低成本存储介质。

4.2 核心数据表设计

系统设计了数十张核心数据表，包括组织机构表、员工信息表、客户信息表、供应商信息表、产品信息表、订单表、发票表、会计科目表、凭证表等。所有数据表都遵循第三范式设计原则，确保数据的一致性和完整性。建立了完善的索引策略，优化查询性能。

第5章 接口设计

系统采用RESTful API设计风格，所有接口都遵循统一的设计规范。接口版本通过URL路径进行控制，如/api/v1/resource。使用HTTP状态码表示请求结果，2xx表示成功，4xx表示客户端错误，5xx表示服务器错误。所有接口都支持JSON格式的请求和响应数据。

接口认证采用JWT Token方案，用户登录后获取访问令牌，后续请求在Header中携带该令牌。接口权限通过RBAC模型进行控制，确保用户只能访问其权限范围内的资源。所有接口都有详细的文档说明，使用Swagger/OpenAPI规范进行描述。

第6章 安全性设计

系统从多个层面进行安全性设计。在应用层，实现了完善的认证和授权机制，防止未授权访问。在数据层，敏感数据采用加密存储，重要操作进行审计日志记录。在网络层，使用HTTPS协议加密传输，部署Web应用防火墙（WAF）防御常见的Web攻击。在系统层，服务器进行安全加固，定期进行漏洞扫描和安全评估。

系统还实现了数据备份和灾难恢复机制，定期进行全量备份和增量备份，确保数据安全。制定了详细的安全应急预案，定期进行安全演练。

第7章 性能优化

系统从多个维度进行性能优化。在前端，采用代码分割、懒加载、缓存等技术提升页面加载速度。在后端，使用多级缓存策略（本地缓存 + 分布式缓存）减少数据库访问。数据库层面，优化SQL语句，建立合适的索引，进行分库分表。架构层面，采用微服务架构，实现服务的独立部署和扩展。

通过CDN加速静态资源访问，使用消息队列实现业务的异步处理。进行容量规划和压力测试，确保系统能够承受预期的业务压力。建立完善性能监控体系，实时监控系统性能指标，及时发现和解决性能问题。